
DEADLOCKS

Deadlocks

- A deadlock is a state where a set of processes request resources that are held by other processes in the set.
 - Assumptions
 - System has only reusable resources
 - Only exclusive access to resources
 - Only one copy of each resource,
- States of a process: running or blocked
- Running state: process has all the resources
 - Blocked state: waiting on one or more resource

Deadlocks

- Resource Deadlocks
 - A process needs multiple resources
 - Deadlock occurs if each process in a set request resources held by another process in the same set, and it must receive all the requested resources to move further.
- Communication Deadlocks
 - Processes wait to communicate with other processes in a set.
 - Each process in the set is waiting on another process's message, and no process in the set initiates a message until it receives a message for which it is waiting.

Deadlock Handling Strategies

- **Deadlock Prevention:** difficult
- **Deadlock Avoidance:** before allocation, check for possible deadlocks.
 - Difficult as it needs global state info in each site (that handles resources).
- **Deadlock Detection:** Find cycles.
Deadlock detection algorithms must satisfy 2 conditions:
 - No undetected deadlocks.
 - No false deadlocks.

Deadlock Handling Strategies

False Deadlocks

- ❑ A **false deadlock** occurs when a deadlock detection algorithm incorrectly identifies a deadlock when there is actually none. This typically happens due to **incorrect or outdated information** in the system.

Causes:

- ❑ Delayed or unsynchronized communication between processes.
- ❑ Incomplete or inconsistent state information in distributed systems.

Phantom Deadlocks

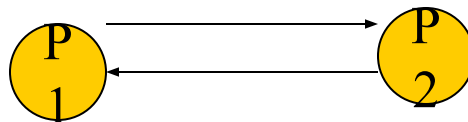
- ❑ A **phantom deadlock** is a special type of false deadlock that arises in **distributed systems** due to **delays in message passing**. It happens when an outdated snapshot of the system shows a deadlock that has already been resolved.

Causes:

- ❑ Delays in message propagation in a distributed system.
- ❑ Inconsistent snapshots of the system state taken at different times.
- ❑ Misinterpretation of process dependencies due to outdated data.

Graph Models

- Nodes of a graph are processes. Edges of a graph the pending requests or assignment of resources.
- Wait-for Graphs (WFG): $P1 \rightarrow P2$ implies $P1$ is waiting for a resource from $P2$.
- Transaction-wait-for Graphs (TWF): WFG in databases.
- Deadlock: directed cycle in the graph.
- Cycle example:



AND, OR Models

□ AND Model

- A process/transaction can simultaneously request for multiple resources.
- Remains blocked until it is granted *all* of the requested resources.

□ OR Model

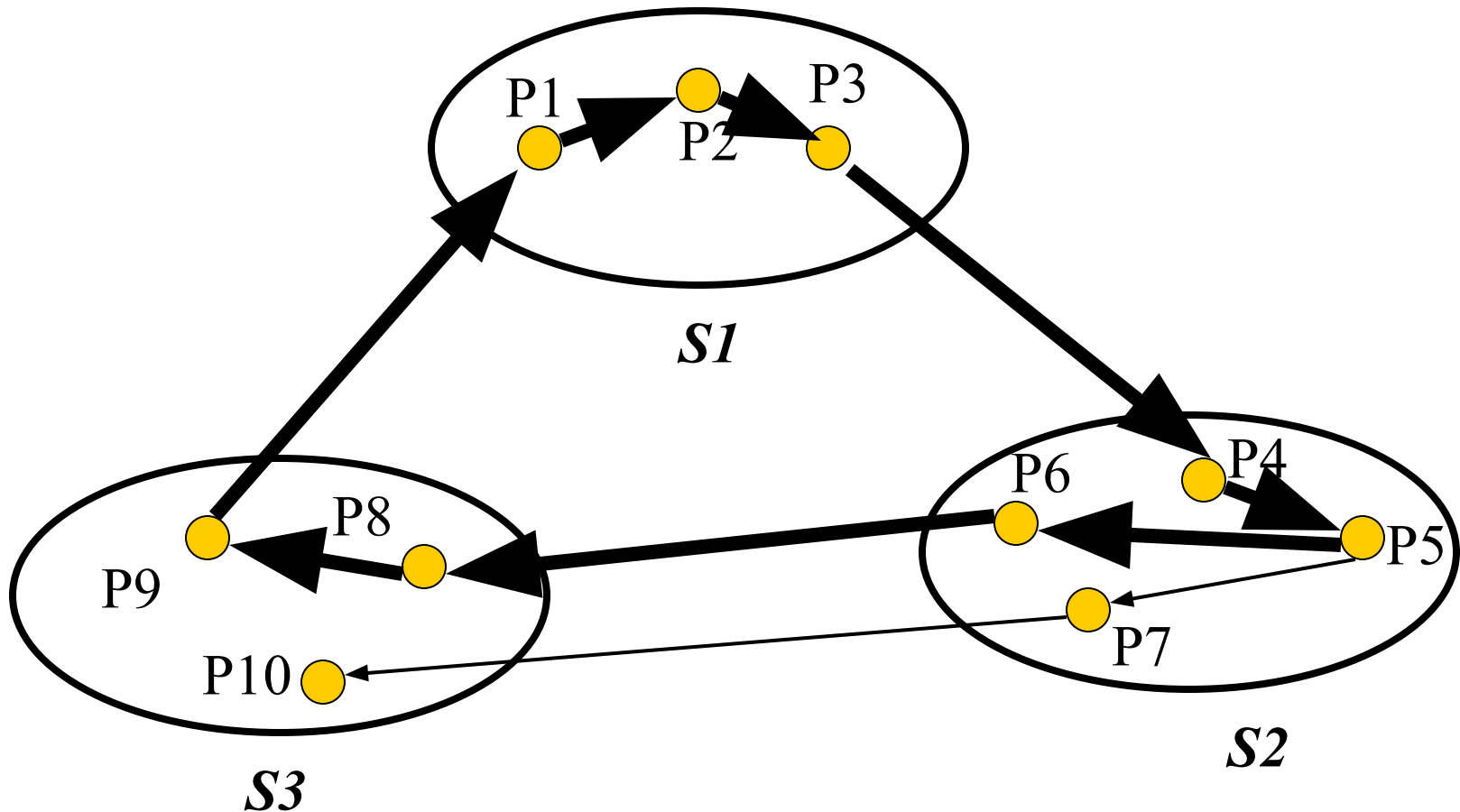
- A process/transaction can simultaneously request for multiple resources.
- Remains blocked till *any one* of the requested resource is granted.

DS Deadlock Detection, Cycle vs. Knot

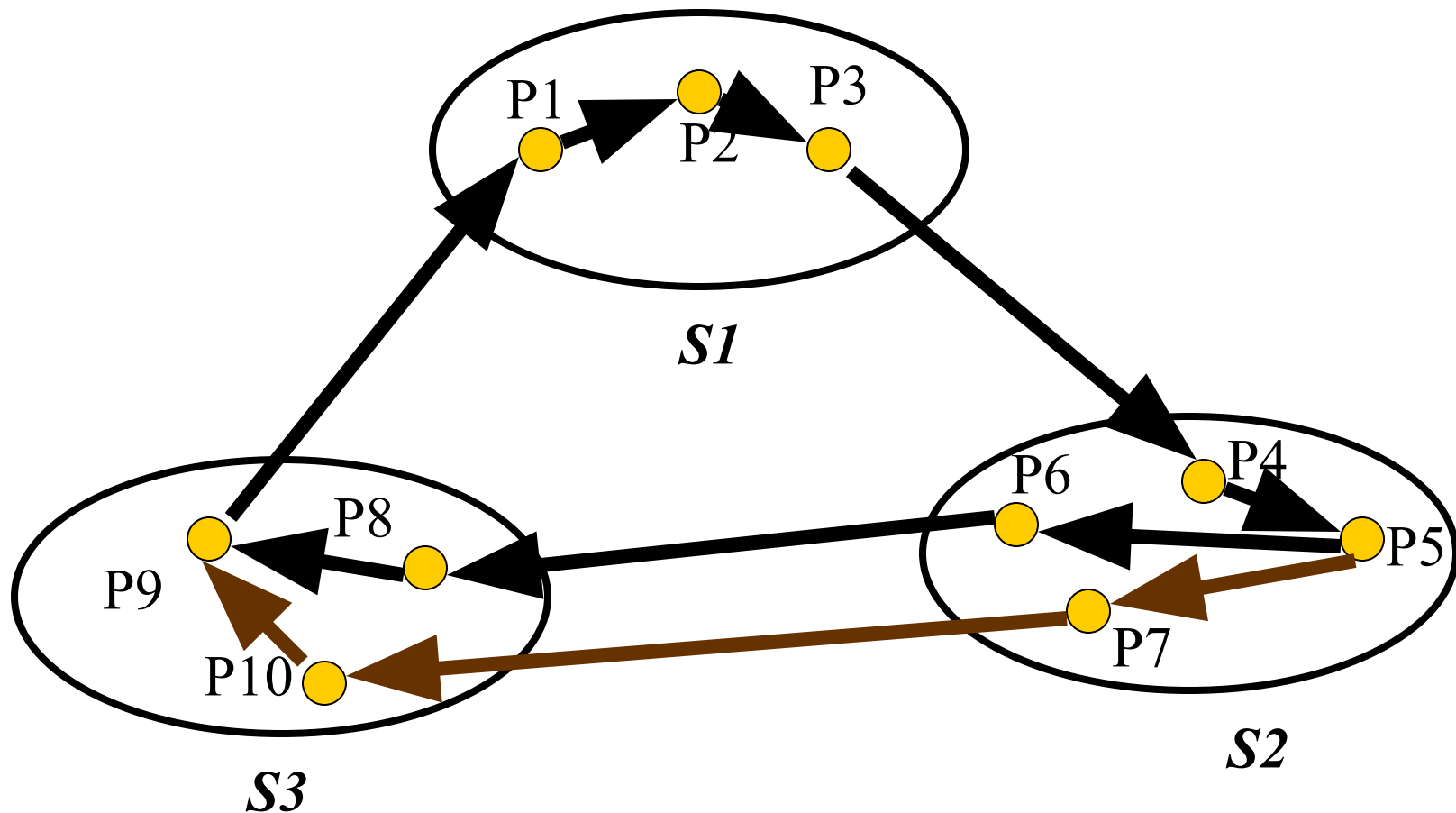
- The AND model of requests requires all resources currently being requested to be granted to un-block a computation
 - A **cycle is sufficient** to declare a deadlock with this model
- The OR model of requests allows a computation making multiple different resource requests to un-block as soon as any are granted
 - A **cycle** is a **necessary** condition
 - A **knot** is a **sufficient** condition

Deadlock in the **AND** model; there is a **cycle**
but no knot

No Deadlock in the **OR** model



Deadlock in both the **AND** model and the **OR** model;
there are **cycles** and a **knot**



DS Detection Requirements

- Progress

- **No undetected deadlocks**

- All deadlocks found
 - Deadlocks found in finite time

- Safety

- **No false deadlock detection**

- Phantom deadlocks caused by network latencies
 - Principal problem in building correct DS deadlock detection algorithms

Distributed Deadlocks

- Centralized Control
 - A *control site* constructs wait-for graphs (WFGs) and checks for directed cycles.
 - WFG can be maintained continuously (or) built on-demand by requesting WFGs from individual sites.
- Distributed Control
 - WFG is spread over different sites. Any site can initiate the deadlock detection process.
- Hierarchical Control
 - Sites are arranged in a hierarchy.
 - A site checks for cycles only in descendents.

Centralized Algorithms

Ho-Ramamoorthy 2-phase Algorithm

Scenario

Consider a distributed system with three sites (S1, S2, S3) and four processes (P1, P2, P3, P4). The processes are holding and waiting for resources in a potential deadlock situation.

- P1 (at S1) holds R1 and waits for R2 (held by P2 at S2).
- P2 (at S2) holds R2 and waits for R3 (held by P3 at S3).
- P3 (at S3) holds R3 and waits for R4 (held by P4 at S1).
- P4 (at S1) holds R4 and waits for R1 (held by P1 at S1).

This forms a circular wait condition, which might indicate a deadlock.

Step-by-Step Execution of Ho-Ramamoorthy 2-Phase Algorithm

Phase 1: Deadlock Detection

1. Each site maintains a status table containing:
 - Locked resources: What resources are held by each process.
 - Waiting resources: What resources are being waited for.
2. Controller requests status tables from all sites periodically.
 - It collects information from S1, S2, and S3.

Centralized Algorithms

Ho-Ramamoorthy 2-phase Algorithm

3. Controller constructs a Wait-for Graph (WFG):

SCSS

P1 → P2 → P3 → P4 → P1 (Cycle detected)

4. Cycle Detected?

- Since a cycle exists, the controller **requests the status tables again**.

Phase 2: False Deadlock Elimination

5. Controller constructs a second WFG from the new status tables.

- This time, it **only includes transactions common in both status tables** (previous & current).
- If the same cycle appears in both, **deadlock is confirmed**.

6. False Deadlock Handling

- If the cycle disappears in the second check, it was a **false deadlock** (possibly due to transient resource requests).
- If the cycle **persists**, the system is **truly in deadlock**.

Centralized Algorithms

Ho-Ramamoorthy 2-phase Algorithm

Deadlock Resolution

7. Victim Selection:

- Assume P1 is chosen for termination to resolve the deadlock.
- P1 releases R1, breaking the cycle.

8. Processes Resume Execution:

- Remaining processes continue execution, and deadlock is avoided.

- **Centralized controller periodically collects status tables.**
- **False deadlocks are filtered out by requesting tables twice.**
- **Deadlocks are resolved by terminating a victim process.**

Illustration of Algorithm Execution

Step 1: Initial Wait-for Graph (WFG)

SCSS

P1 → P2 → P3 → P4 → P1 (Cycle detected)

Step 2: Second WFG Construction

- If the same cycle appears again, **deadlock confirmed**.
- If the cycle disappears, it was a **false deadlock**.

Step 3: Deadlock Resolution

- Terminate P1 → Releases R1.
- Break cycle → Deadlock avoided.

Centralized Algorithms

□ **Ho-Ramamoorthy 2-phase Algorithm**

- Each site maintains a status table of all processes initiated at that site: includes all resources locked & all resources being waited on.
- Controller requests (periodically) the status table from each site.
- Controller then constructs WFG from these tables, searches for cycle(s).

-
- If no cycles, no deadlocks Otherwise, (cycle exists): Request for state tables again.
 - Construct WFG based *only* on common transactions in the 2 tables.
 - If the same cycle is detected again, system is in deadlock.

Later proved: cycles in 2 consecutive reports *need not* result in a deadlock. Hence, this algorithm detects false deadlocks.

-
- Why are centralized algorithms not favorable ?

Distributed Algorithms

- ▣ **Path-pushing:** resource dependency information disseminated through designated paths (in the graph).
- ▣ **Edge-chasing:** special messages or probes circulated along edges of WFG. Deadlock exists if the probe is received back by the initiator.
- ▣ **Diffusion computation:** queries on status sent to process in WFG.
- ▣ **Global state detection:** get a snapshot of the distributed system.

Path-pushing Algorithm

- **Obermarck's Algorithm:** used for databases. Transactions lock and wait on resources. One transaction can initiate atmost one sub-transaction at a given point in time.
 - A site waits for deadlock-related information from other sites.
 - The site combines the received information and local graph to build an updated (global) graph.
 - Non-local portion of graph: distinguished by nodes called *Ex* (External).

Obermarck's Path-Pushing Algorithm

Detection Process

Case 1: If Site S_n finds a cycle not involving 'ex', Deadlock exists.

Case 2: If Site S_n finds a cycle involving 'ex', Deadlock possible.

- If Case 2 ->

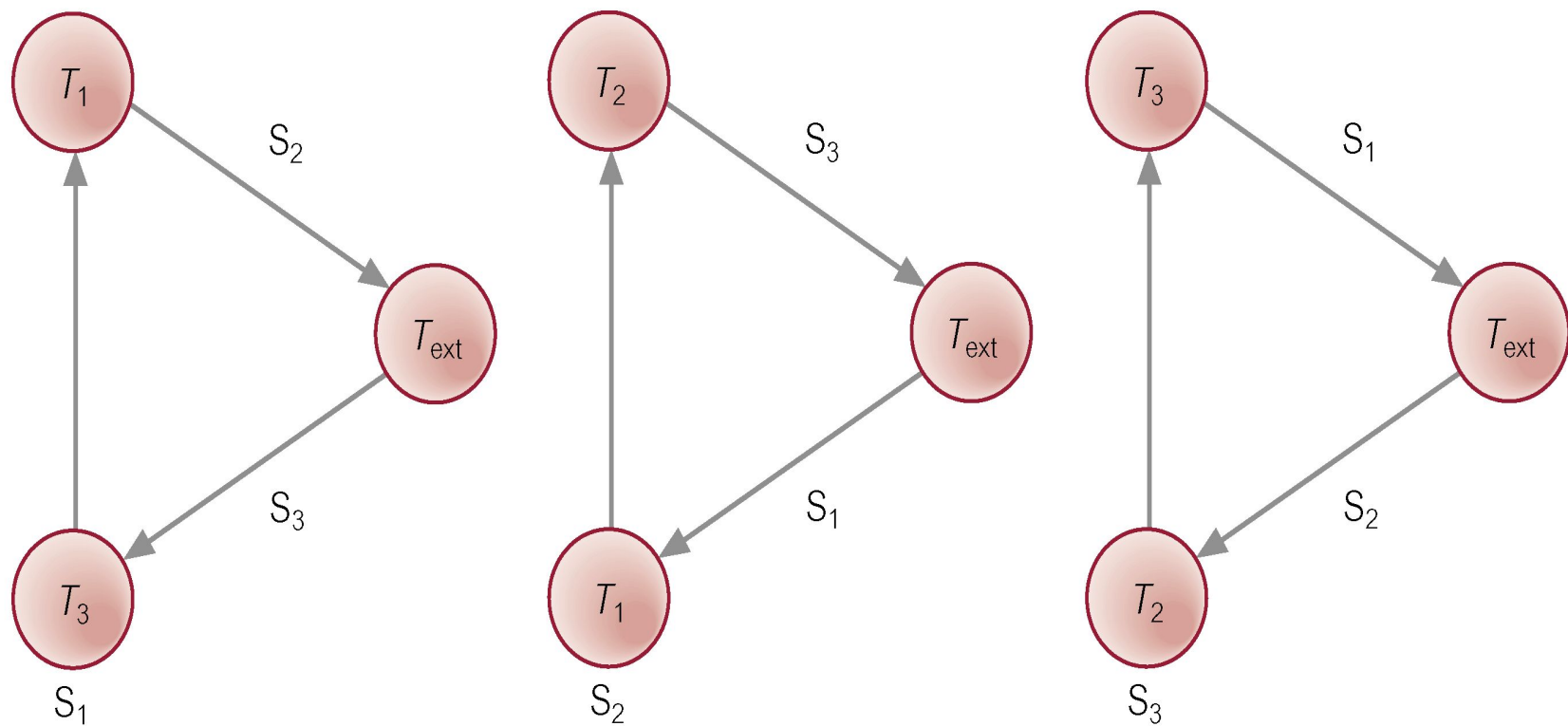
Site S_n sends a message containing its detected cycles to other sites. All sites receive the message, update their WFG and re-evaluate the graph.

□ Consider Site S_j receives the message:

- Site S_j checks for local cycles. If cycle found not involving 'ex' (of S_j) -> Deadlock exists.
- If site S_j finds cycle involving 'ex' it forwards the message to other sites.

Process continues till deadlock found.

Distributed Deadlock Detection



Distributed Deadlock Detection

$$S_1: T_{\text{ext}} \rightarrow T_3 \rightarrow T_1 \rightarrow T_{\text{ext}}$$

$$S_2: T_{\text{ext}} \rightarrow T_1 \rightarrow T_2 \rightarrow T_{\text{ext}}$$

$$S_3: T_{\text{ext}} \rightarrow T_2 \rightarrow T_3 \rightarrow T_{\text{ext}}$$

- Transmit LWFG for S_1 to the site for which transaction T_1 is waiting, site S_2 .
- LWFG at S_2 is extended and becomes:

$$S_2: T_{\text{ext}} \rightarrow T_3 \rightarrow T_1 \rightarrow T_2 \rightarrow T_{\text{ext}}$$

Distributed Deadlock Detection

- Still contains potential deadlock, so transmit this WFG to S_3 :

$$S_3: T_{\text{ext}} \rightarrow T_3 \rightarrow T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_{\text{ext}}$$

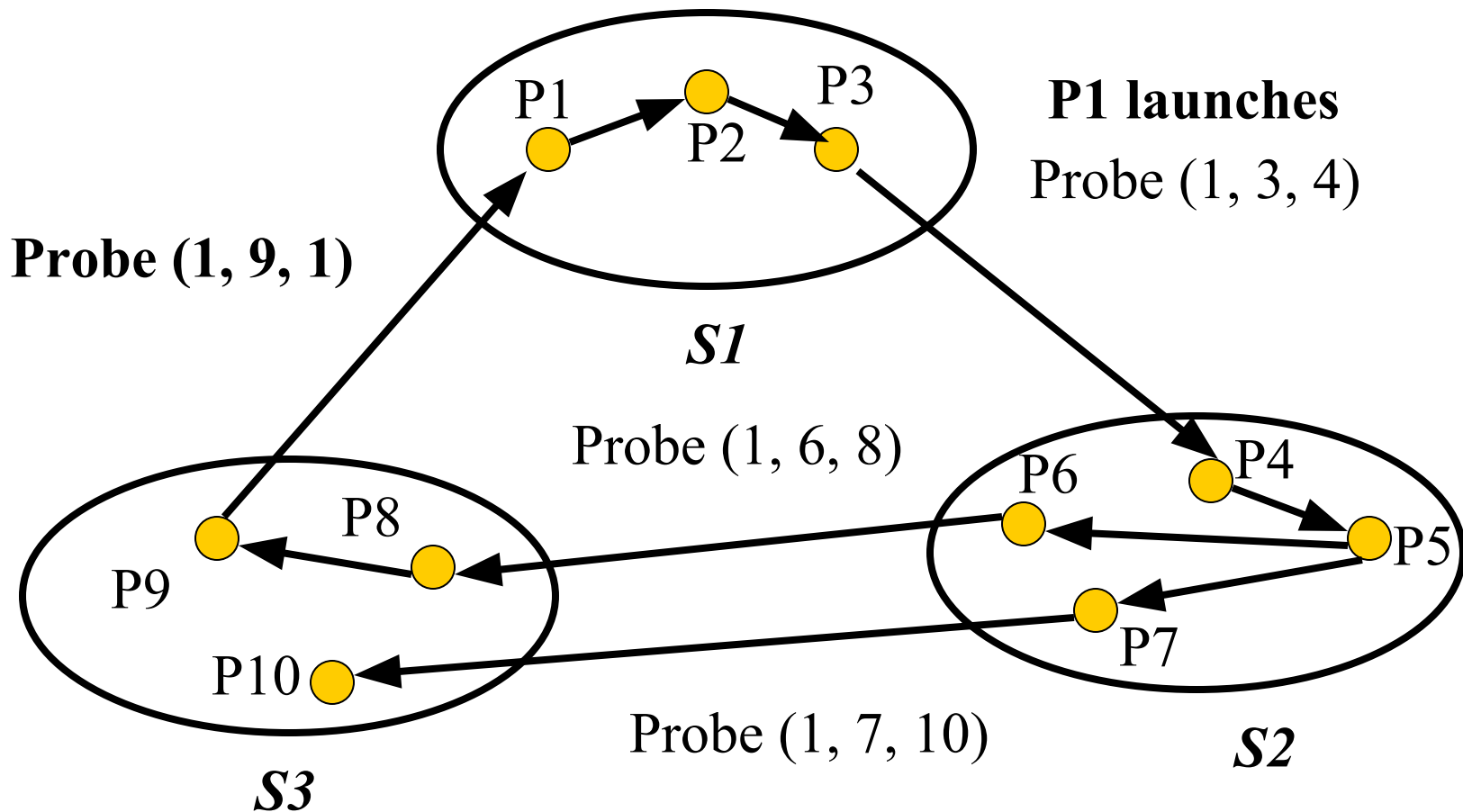
- GWFG contains cycle not involving T_{ext} , so deadlock exists.

-
- ▣ **Problems:** Obermarck's detect false deadlocks
-> snapshot of the distributed system taken asynchronously by different sites. Global cycles can change with time and may not be reflected in the local information.

Edge Chasing Algorithms

- Chandy-Misra-Haas Algorithm (an **AND** model)
 - probe messages $M(i, j, k)$
 - initiated by P_j for P_i and sent to P_k
 - probe messages work their way through the WFG and if they return to sender, a deadlock is detected

Chandy-Misra-Haas Algorithm



Chandy-Misra-Haas's Algorithm:

- A probe(i, j, k) is used by a deadlock detection process P_i . This probe is sent by the home site of P_j to P_k .
 - This probe message is circulated via the edges of the graph. Probe returning to P_i implies deadlock detection.
 - Terms used:
 - P_j is *dependent* on P_k , if a sequence of $P_j, P_{i1}, \dots, P_{im}, P_k$ exists.
 - P_j is *locally dependent* on P_k , if above condition + P_j, P_k on same site.
 - Each process maintains an array *dependenti*: *dependenti(j)* is true if P_i knows that P_j is dependent on it. (initially set to false for all i & j).

Chandy-Misra-Haas's Algorithm

Sending the probe:

if P_i is locally dependent on itself then deadlock.

else for all P_j and P_k such that

(a) P_i is locally dependent upon P_j , and

(b) P_j is waiting on P_k , and

(c) P_j and P_k are on different sites, send $\text{probe}(i,j,k)$ to the home site of P_k .

Receiving the probe:

if (d) P_k is blocked, and

(e) $\text{dependent}_k(i)$ is false, and

(f) P_k has not replied to all requests of P_j ,

then begin

$\text{dependent}_k(i) := \text{true};$

if $k = i$ then P_i is deadlocked else ...

Chandy-Misra-Haas's Algorithm

Receiving the probe:

.....

else for all P_m and P_n such that

(a') P_k is locally dependent upon P_m , and

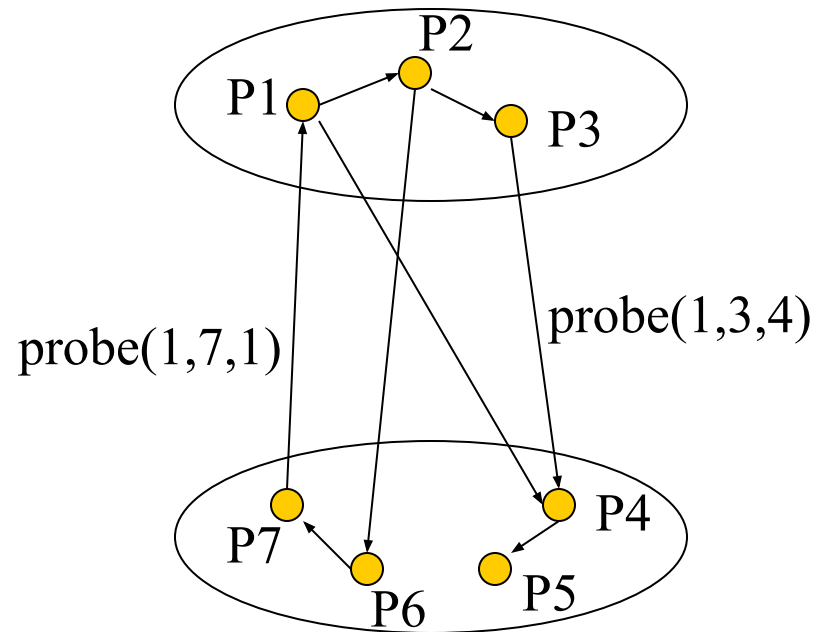
(b') P_m is waiting on P_n , and

(c') P_m and P_n are on different sites, send

probe(i, m, n) to the home site of P_n .

end.

C-M-H Algorithm: Example



Deadlock Resolution

- Efficient resolution of deadlock requires knowledge of all processes and resources involved in the deadlock
- in many algorithms more than one processes may detect the same deadlock
- Race Condition:
 - Two processes block at the same time and send probes
 - Both discover the cycle in parallel

Solution

- use of process priorities can help to address these issues.. consider priorities introduced in Obermarck's algorithm.
- Highest priority process initiates and detects deadlock (initiations by lower priority ones are suppressed).

Resolution

- When deadlock is detected, lowest priority process(es) can be aborted to resolve the deadlock.
- Resolving a deadlock requires that
 - victim is terminated and resources restored to a proper state
 - detection info for victim flushed out from the system quickly since otherwise phantom deadlocks may be detected
 - detection and resolution interact extensively

Differences between

□ deadlock & starvation

A Deadlock detection algorithm based on global state detection

Assignment

Consider a distributed system with three sites (S1, S2, S3) and four processes (P1, P2, P3, P4). The processes are holding and waiting for resources in a potential deadlock situation.

- P1 (at S1) holds R1 and waits for R2 (held by P2 at S2).
- P2 (at S2) holds R2 and waits for R3 (held by P3 at S3).
- P3 (at S3) holds R3 and waits for R4 (held by P4 at S1).
- P4 (at S1) holds R4 and waits for R1 (held by P1 at S1).

Questions

- **(a)** Construct the Wait-For Graph (WFG) using the status tables collected at time T1.
- **(b)** Is a cycle present in the WFG at T1? Justify.
- **(c)** According to the Ho-Ramamoorthy 2-Phase algorithm, explain why the controller must collect the status tables again.
- **(d)** Given that the second snapshot (T2) is identical to the first, determine whether the system is in real deadlock or false deadlock. Justify your answer.